Midterm Activity 5: Identifying Good and Bad Coding Practices

Software Engineering Project Evaluation & Code Audit — **DinkMatch Kiosk System**

Course/Activity: Midterm Activity 5 (Group Work)

Document Format: Legal Size (8.5" × 14")

Project Title: DinkMatch (Local-First Matchmaking System)

Target Repository: DinkMatch-Thesis

Institution: Don Mariano Marcos Memorial State University

Evaluation Date: August 17, 2026

Evaluating Researchers (Thesis Team): Jericho T. Boado (Leader), Joana Andrae J. Agbunag, Chelsie Assyria P. Boac, Maerose Joscel Czarinah V. Boadilla, Karina Joyce B. Geneta, Darmie A. Sison

Academic Scope: Software Engineering Code Quality Audit

1. Project Overview & Setup Instructions

DinkMatch is a local-first touchscreen matchmaking kiosk and zero-sum rating calibration engine designed for racket sport facilities. The codebase is structured in a clean 2-folder repository layout: dinkmatch-app/ (Vue 3 / Quasar PWA application) and docs/ (Technical documentation & architecture assets). To enable all group members and evaluators to run and audit the software on their local devices, follow the verified setup instructions below:

1	`cd dinkmatch-app`	Navigate into the primary web application directory.
2	`npm install`	Installs all Quasar, Vue 3, Vite, and edge-tts dependencies.
3	`npm run dev`	Launches the local Vite hot-reloading server on https://localhost:9000/.
4	Open Browser at https://localhost:9000/	Access the interactive PWA kiosk. (HTTPS is required for basicSSL & PWA service workers; accept the self-signed cert prompt).

2. Professional Coding Standards and Practices

The dinkmatch-app codebase adheres strictly to modern TypeScript and Vue 3 industry standards. The following table highlights key professional practices implemented across the system:

Standard / Practice	Code Location & Evidence	Implementation Analysis
Strict TypeScript Typing	src/services/matchmaking.ts (Lines 10-134)	Explicit data interfaces (Player, QueueEntry, ActiveMatch, AppState) eliminate implicit any types and enable strict compile-time checking.
Single Responsibility Principle (SRP)	src/services/ vs src/composables/	Clear architectural separation: matchmaking.ts handles pure mathematical Elo calculations, composables handle reactive state, and Vue components handle UI layout.
Automated Style & Format Enforcement	.eslintrc.cjs & package.json	Configured ESLint and Prettier formatting pipeline (npm run lint, npm run format) ensuring zero style deviations across developers.
Modular Vue 3 Composition API	src/pages/DeleteAccountPage.vue (Lines 157-195)	Uses <script setup lang="ts"> with reactive ref and computed primitives, producing readable and highly performant component logic.
Upper Snake Case Constants	src/services/matchmaking.ts (Lines 389-405)	Domain mathematical constants (K_DOUBLES = 64, MARGIN_WEIGHT = 0.15, NOVELTY_WEIGHTS) are declared immutable and named according to standard conventions.

3. Evaluation Against the 15 Principles of Quality Software

Quality software is evaluated across 15 foundational software engineering principles. Below is the comprehensive audit matrix mapping each principle directly to concrete implementations within the DinkMatch repository:

1. Maintainability	src/services/matchmaking.ts (Lines 141-158)	Single source of truth CLUB_SETTINGS mapping all configurable system defaults in one dictionary.
2. Readability	src/services/matchmaking.ts (Lines 509-532)	Self-documenting pure utility functions (computeHarmonicMean, computeArithmeticMean, computeTeamRating).
3. Reusability	src/components/PlayerAvatar.vue	Encapsulated player avatar rendering component reusable across leaderboard, queue, and match dialogs.
4. Reliability	src/services/matchmaking.ts (Lines 164-172)	Offline-first LocalStorage state persistence and 7-day tombstone garbage collection (gcTombstones).
5. Efficiency	quasar.config.ts (Lines 60-77)	Vite build optimizations with basicSSL and custom rollup manual chunks splitting vendor dependencies.
6. Robustness	src/services/matchmaking.ts (Lines 467-483)	Anti-carry partner gap penalty and rating floor (Math.max(RATING_FLOOR, ...)) preventing negative or corrupted player ratings.
7. Usability	src/pages/DeleteAccountPage.vue (Lines 18-25)	Informative user feedback banners, dense form fields with inline validation rules, and responsive modal cards.
8. Security	src/services/likhaClient.ts (Lines 15-27)	Custom QuasarStorage adapter isolating authentication tokens in browser LocalStorage without cross-site cookie vulnerabilities.
9. Portability	dinkmatch-app/package.json (Lines 80-84)	Multi-target execution supporting Web PWA, Desktop Browser, and Android TWA (Bubblewrap APK bundle).
10. Testability	src/services/matchmaking.ts (Lines 433-507)	Pure functional RatingEngine.calculateShift accepts parameters and returns deterministic results without side effects.
11. Scalability	src/services/likhaClient.ts (Lines 31-46)	Decoupled SDK client with WebSocket real-time synchronization supporting expansion from single kiosk to multi-club networks.
12. Correctness	src/services/matchmaking.ts (Lines 405-420)	Largest-remainder integer allocation (allocateInteger) guaranteeing exact zero-sum rating conservation ($\sum \Delta R = 0$).
13. Modularity	src/composables/useNotify.ts	Decoupled composable isolating UI toast notification triggers from business domain services.
14. Flexibility	src/services/matchmaking.ts (Lines 575-655)	Configurable matchmaking engine supporting 5 distinct heuristic modes (strict_balance, variety_first, balance_first, etc.).
15. Simplicity	src/services/matchmaking.ts (Lines 551-573)	Non-recursive combinatorial pairing generator (getCombinations) computing 2v2 permutations cleanly.

4. Defensive Programming Implementations

Defensive programming anticipates potential edge cases, invalid user input, and unexpected state corruption. The DinkMatch system implements several critical defensive guard logic mechanisms:

1. Concurrency & State Invariant Guards (One Match Per Player & Court)

Location: src/services/matchmaking.ts (Lines 174-242 & 246-312)

Before serializing local state to storage (saveState), the system defensively validates active matches. If a player or court appears in multiple active matches due to rapid multi-touch interaction or sync race conditions, losing matches are tombstoned and players returned to queue.

```
enforceOneMatchPerPlayerOnState(state: AppState);
enforceOneMatchPerCourtOnState(state: AppState);
```

2. Concurrency Capacity Cap Demotion

Location: src/services/matchmaking.ts (Lines 319-360)

If the count of in-progress matches exceeds physical court capacity (availableCourts), excess matches are automatically demoted to 'waiting' status without dropping player assignments.

```
if (inProgress.length > cap) {
  const toDemote = inProgress.slice(cap);
  toDemote.forEach(m => m.status = 'waiting');
}
```

3. Partner Ratio Weight Capping

Location: src/services/matchmaking.ts (Lines 425-431)

To prevent extreme skill rating disparities from causing disproportionate rating transfers during match outcome processing, teammate weights are defensively capped at a max ratio of 2.0.

```
const capWeights = (weights: number[], maxRatio: number): number[] => {
  const max = Math.max(...weights);
  const minAllowed = max / maxRatio;
  return weights.map((w) => Math.max(w, minAllowed));
};
```

5. Error and Exception Handling

Robust error exception handling ensures that runtime exceptions fail gracefully without corrupting state or halting the kiosk interface.

Global 401 Unauthorized Token Interceptor & Race Condition Prevention

Location: src/services/likhaClient.ts (Lines 52-75)

Handles token expiration (e.g., kiosk wake from sleep). Intercepts 401 exceptions, memoizes concurrent refresh requests via _refreshing promise, retries the original request upon success, or throws original 401 if refresh fails.

```
let _refreshing: Promise | null = null;
const originalRequest = _client.request.bind(_client);
type RequestOptions = Parameters[0];
_client.request = (async (options: RequestOptions): Promise => {
  try {
    return (await originalRequest(options)) as T;
  } catch (err: unknown) {
    const error = err as { response?: { status?: number } };
    if (error?.response?.status !== 401) throw err;
    if (!_refreshing) _refreshing = _client.refresh().finally(() => { _refreshing = null; });
    await _refreshing;
    return (await originalRequest(options)) as T;
  }
}) as typeof _client.request;
```

Async Form State Protection with Try-Finally Blocks

Location: src/pages/DeleteAccountPage.vue (Lines 170-195)

Form submission handlers execute inside a try-finally block, guaranteeing that the loading.value state variable resets to false even if network requests throw uncaught exceptions.

```
const onSubmit = async () => {
  loading.value = true;
  try {
    await likhaClient.request({ path: '/items/account_deletions', method: 'POST', body: ... });
  } catch {
    console.warn('[AccountDeletion] Recorded local request.');
  } finally {
    loading.value = false; // Guarantees UI button re-enables
  }
};
```

6. Identification of Bad Coding Practices & Anti-Patterns

To satisfy the academic activity objective of identifying code areas that do not fully adhere to ideal software engineering principles, the codebase was audited to document concrete anti-patterns that currently remain in the project:

Anti-Pattern #1: Unsanitized Production Console Logging Telemetry

Category: Production Telemetry Pollution & Performance Degradation

File Location: src/services/matchmaking.ts (Lines 239, 309, 355) & src/services/playerProfile.ts (Lines 210-221)

Current Implementation (Unrefactored Code in Repository):

```
console.log(`[enforceOneMatchPerPlayer] Removed ${matchesToRemove.size} match(es)...`);
console.log('[PlayerProfile] Raw matches response:', matches);
```

Technical Impact & Retention Context:

Excessive console logging pollutes the browser console and degrades performance on low-power kiosk hardware. Retained intentionally for kiosk debugging and live verification during thesis defense demonstrations.

Recommended Future Refactoring: Encapsulate logging behind an environment check (e.g., `if (import.meta.env.DEV) console.log(...)`) to sanitize production builds.

Anti-Pattern #2: Direct Private Library Buffer Mutation & Third-Party Type Suppression

Category: Encapsulation Violation & Third-Party Type Suppression

File Location: src/services/edgeTts.ts (Lines 1 & 28)

Current Implementation (Unrefactored Code in Repository):

```
// @ts-expect-error - no type declarations shipped with package
import EdgeTTSBrowser from '@kingdanx/edge-tts-browser';

// Reset accumulated audio buffer (library doesn't do this between calls)
tts.file = new Uint8Array();
```

Technical Impact & Retention Context:

Directly mutating the internal `tts.file` byte buffer violates object encapsulation principles and risks breakage if the underlying library updates its private property schema. Retained as a pragmatic patch for continuous voice announcements.

Recommended Future Refactoring: Wrap the speech synthesis module in a custom facade class that encapsulates byte stream cleanup without directly mutating library internals.

Anti-Pattern #3: Loose Data Coupling via Imperative Any Assertions in Legacy Sync Helpers

Category: Weak Type Safety & Unchecked Structural Payload Assumptions

File Location: src/services/cloudSyncHelpers.ts (Lines 18-35)

Current Implementation (Unrefactored Code in Repository):

```
export function mapRemoteToLocalPlayer(remote: any): Player {
  return {
    username: remote.username,
    rating: remote.rating || 1200,
  } as Player;
}
```

Technical Impact & Retention Context:

Accepting `remote: any` bypasses TypeScript compiler checks, allowing unexpected remote API payload changes to propagate runtime `undefined` values into local state. Retained for database schema tolerance during sync.

Recommended Future Refactoring: Implement Zod or Valibot schema validation to parse and validate incoming remote synchronization payloads before casting to local state models.

Anti-Pattern #4: Catch-and-Log Error Swallowing in Auxiliary Kiosk Services

Category: Silent Exception Swallowing Anti-Pattern

File Location: src/services/playerReport.ts (Lines 174, 228, 348)

Current Implementation (Unrefactored Code in Repository):

```
try {
  await sendReportPayload();
} catch (err) {
  console.error('submitPlayerReport error:', err);
}
```

Technical Impact & Retention Context:

Catching errors and only logging them to console prevents callers from detecting network failures, leaving the user without confirmation or retry prompts. Retained to prevent auxiliary network failures from crashing the kiosk UI.

Recommended Future Refactoring: Propagate handled errors back to the UI component to display retry toasts and record queued sync requests in LocalStorage.

7. Summary and Audit Conclusion

The software quality evaluation of the **DinkMatch** repository confirms that the application exhibits strong architectural discipline, satisfying all 15 principles of quality software, strict TypeScript typing, and defensive state guards. Documenting the remaining unrefactored anti-patterns (such as development telemetry logging, third-party buffer mutations, and permissive sync payload mapping) provides a clear technical roadmap for post-defense software optimization.

Evaluated & Submitted By (Student Researchers / Thesis Proponents):

- **Jericho T. Boado** (Group Leader)
- **Joana Andrae J. Agbunag**
- **Chelsie Assyria P. Boac**
- **Maerose Joscel Czarinah V. Boadilla**
- **Karina Joyce B. Geneta**
- **Darmie A. Sison**

Academic Institution & Program:

Don Mariano Marcos Memorial State University
College of Computer Science
Bachelor of Science in Computer Science
Software Engineering Course Project